

Module 11: GROUP BY Clause

The **GROUP BY** clause is used to group rows that have the same values into summary rows. It is one of the most important concepts in SQL analytics because it enables category-wise calculations such as total sales by city, average salary by department, and customer counts by region.



Introduction

Aggregate functions like:

```
COUNT()  
SUM()  
AVG()  
MIN()  
MAX()
```

calculate values across all rows.

But what if management asks:

- Total salary by department
- Number of employees in each city
- Average sales by product category
- Revenue by month

To answer these questions, we use:

```
GROUP BY
```



Learning Objectives

After completing this module, you will be able to:

- ✓ Understand GROUP BY
- ✓ Group records by one column
- ✓ Group records by multiple columns
- ✓ Use aggregate functions with GROUP BY
- ✓ Differentiate GROUP BY and DISTINCT

- ✓ Build analytical reports
- ✓ Avoid common grouping errors



Table of Contents

1. What is GROUP BY?
2. Why GROUP BY is Needed
3. GROUP BY Syntax
4. COUNT with GROUP BY
5. SUM with GROUP BY
6. AVG with GROUP BY
7. MIN and MAX with GROUP BY
8. Multiple Aggregate Functions
9. GROUP BY Multiple Columns
10. GROUP BY vs DISTINCT
11. Query Execution Order
12. Common Errors
13. Best Practices
14. Business Analytics Examples
15. Summary
16. Practice Questions



1 What is GROUP BY?

GROUP BY groups rows having the same value into a single group.

Aggregate functions then operate on each group separately.

Example Table

EmployeeID	Department	Salary
101	IT	50000
102	IT	60000
103	HR	45000
104	HR	55000

Question:

What is the average salary by department?

Using GROUP BY:

```
SELECT Department,  
       AVG(Salary)  
FROM Employees  
GROUP BY Department;
```

Output

Department	AvgSalary
IT	55000
HR	50000

Why GROUP BY Matters

Used in:

- Dashboards
- KPI Reports
- Revenue Analysis
- Sales Reports
- Customer Segmentation

2 Why GROUP BY is Needed

Without GROUP BY:

```
SELECT AVG(Salary)  
FROM Employees;
```

Output:

52500

One overall average.

With GROUP BY:

```
SELECT Department,  
       AVG(Salary)  
FROM Employees  
GROUP BY Department;
```

Output:

```
Department-wise averages
```

Real-World Questions

Question	GROUP BY Column
Revenue by City	City
Sales by Product	Product
Employees by Department	Department
Orders by Month	Month

3 GROUP BY Syntax

Basic syntax:

```
SELECT ColumnName,  
       AggregateFunction(ColumnName)  
FROM TableName  
GROUP BY ColumnName;
```

Example

```
SELECT Department,  
       COUNT(*)
```

```
FROM Employees  
GROUP BY Department;
```

General Structure

```
SELECT  
FROM  
WHERE  
GROUP BY  
HAVING  
ORDER BY
```



COUNT() with GROUP BY

Count rows in each group.

Example Table

Department
IT
IT
HR
HR
HR

Query:

```
SELECT Department,  
       COUNT(*) AS EmployeeCount  
FROM Employees  
GROUP BY Department;
```

Output:

Department	EmployeeCount
IT	2
HR	3

Business Uses

- Employees per department
- Customers per city
- Orders per month

5 SUM() with GROUP BY

Calculate totals within each group.

Example

Department	Salary
IT	50000
IT	60000
HR	45000
HR	55000

Query:

```
SELECT Department,  
       SUM(Salary) AS TotalSalary  
FROM Employees  
GROUP BY Department;
```

Output:

Department	TotalSalary
IT	110000
HR	100000

Business Uses

- Revenue by City
- Sales by Product
- Salary Expense by Department

6 AVG() with GROUP BY

Calculate averages for each group.

Query:

```
SELECT Department,  
       AVG(Salary) AS AverageSalary  
FROM Employees  
GROUP BY Department;
```

Output:

Department	AverageSalary
IT	55000
HR	50000

Business Uses

- Average Salary
- Average Order Value
- Average Revenue

7 MIN() and MAX() with GROUP BY

Find smallest and largest values within groups.

MAX Example

```
SELECT Department,  
       MAX(Salary) AS HighestSalary
```

```
FROM Employees
GROUP BY Department;
```

Output:

Department	HighestSalary
IT	60000
HR	55000

MIN Example

```
SELECT Department,
       MIN(Salary) AS LowestSalary
FROM Employees
GROUP BY Department;
```

Output:

Department	LowestSalary
IT	50000
HR	45000

8 Multiple Aggregate Functions

Use several aggregates in one query.

Example

```
SELECT Department,
       COUNT(*) AS TotalEmployees,
       SUM(Salary) AS TotalSalary,
       AVG(Salary) AS AvgSalary,
       MIN(Salary) AS MinSalary,
       MAX(Salary) AS MaxSalary
```

```
FROM Employees
GROUP BY Department;
```

Output

Department	Employees	Total	Avg	Min	Max
IT	2	110000	55000	50000	60000
HR	2	100000	50000	45000	55000



GROUP BY Multiple Columns

Group using more than one column.

Example Table

City	Department	Salary
Delhi	IT	50000
Delhi	HR	40000
Mumbai	IT	60000
Mumbai	HR	55000

Query

```
SELECT City,
       Department,
       AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY City,
       Department;
```

Output

City	Department	AvgSalary
Delhi	IT	50000
Delhi	HR	40000
Mumbai	IT	60000

City	Department	AvgSalary
Mumbai	HR	55000

How SQL Groups

```
Delhi + IT
Delhi + HR
Mumbai + IT
Mumbai + HR
```

Each combination becomes a group.

10 GROUP BY vs DISTINCT

Both remove duplication but serve different purposes.

DISTINCT

Returns unique values.

Example:

```
SELECT DISTINCT Department
FROM Employees;
```

Output:

```
IT
HR
```

GROUP BY

Creates groups for calculations.

Example:

```
SELECT Department,  
       COUNT(*)  
FROM Employees  
GROUP BY Department;
```

Output:

```
IT → 2  
HR → 3
```

Comparison

Feature	DISTINCT	GROUP BY
Removes duplicates	✓	✓
Uses Aggregates	✗	✓
Creates Groups	✗	✓

Query Execution Order

Query:

```
SELECT Department,  
       AVG(Salary)  
FROM Employees  
WHERE Salary > 40000  
GROUP BY Department;
```

Actual Execution:

1. FROM
2. WHERE
3. GROUP BY
4. SELECT
5. ORDER BY

Understanding execution order helps:

- Debug queries
- Optimize performance
- Learn HAVING

1 2 Common Errors

Selecting Non-Grouped Columns

Bad:

```
SELECT Department,  
       EmployeeName,  
       AVG(Salary)  
FROM Employees  
GROUP BY Department;
```

Error:

```
EmployeeName is not grouped
```

Correct

```
SELECT Department,  
       AVG(Salary)  
FROM Employees  
GROUP BY Department;
```

Forgetting GROUP BY

Bad:

```
SELECT Department,  
       AVG(Salary)  
FROM Employees;
```

Error.

Wrong Aggregate Usage

Mixing grouped and non-grouped columns incorrectly.

1 **3** Best Practices

Use Meaningful Aliases

Good:

```
AVG(Salary) AS AverageSalary
```

Group Only Required Columns

Avoid unnecessary grouping.

Filter Early

Use:

```
WHERE
```

before grouping.

Use Multiple Aggregates

Improves efficiency.

Format Queries Properly

Readable SQL is maintainable SQL.

Professional Example

```
SELECT Department,  
  
       COUNT(*) AS EmployeeCount,  
  
       AVG(Salary) AS AverageSalary,  
  
       MAX(Salary) AS HighestSalary  
  
FROM Employees  
GROUP BY Department;
```

Business Analytics Examples

Employees by Department

```
SELECT Department,  
       COUNT(*)  
FROM Employees  
GROUP BY Department;
```

Revenue by Product

```
SELECT ProductName,  
       SUM(SalesAmount)  
FROM Sales  
GROUP BY ProductName;
```

Orders by Month

```
SELECT MONTH(OrderDate),  
       COUNT(*)  
FROM Orders  
GROUP BY MONTH(OrderDate);
```

Customer Count by City

```
SELECT City,  
       COUNT(*)  
FROM Customers  
GROUP BY City;
```

KPI Dashboard Query

```
SELECT Department,  
       COUNT(*) AS Employees,  
       AVG(Salary) AS AvgSalary,  
       SUM(Salary) AS TotalSalary  
FROM Employees  
GROUP BY Department;
```



Summary

In this module, you learned:

- ✓ GROUP BY
- ✓ COUNT with GROUP BY
- ✓ SUM with GROUP BY
- ✓ AVG with GROUP BY
- ✓ MIN with GROUP BY
- ✓ MAX with GROUP BY
- ✓ Multiple Aggregates
- ✓ Multi-Column Grouping
- ✓ GROUP BY vs DISTINCT
- ✓ Query Execution Order



Practice Questions

Theory

1. What is GROUP BY?
 2. Why is GROUP BY needed?
 3. How does GROUP BY work?
 4. Difference between GROUP BY and DISTINCT?
 5. Can GROUP BY use multiple columns?
 6. Why do we use aggregate functions with GROUP BY?
 7. What happens if a column is not grouped?
 8. What is the execution order involving GROUP BY?
 9. How does COUNT work with GROUP BY?
 10. How does AVG work with GROUP BY?
-

Practical Exercises

Task 1

Count employees by department.

Task 2

Calculate total salary by department.

Task 3

Calculate average salary by city.

Task 4

Find highest salary in each department.

Task 5

Find lowest salary in each department.

Task 6

Use multiple aggregates in one query.

Task 7

Group by:

City
Department

simultaneously.

Challenge Project

Create an Employee Analytics Dashboard query showing:

- Department
- Total Employees
- Average Salary
- Highest Salary
- Lowest Salary
- Total Salary Expense

using a single GROUP BY query.



Next Module

→ Module 12: HAVING Clause

Topics Covered:

- What is HAVING?
- HAVING vs WHERE
- Filtering Groups
- HAVING with COUNT()
- HAVING with SUM()
- HAVING with AVG()
- HAVING with Multiple Conditions
- Real-World Analytics Queries